Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

23. Depuración

Todo lo de este tema es de GHC y `ghci`, no del estándar Haskell.

Avisos

- `:set -Wall` en `ghci` (o `ghci -Wall archivo.hs`) activa casi todos los avisos.
  - Patrones no exhaustivos, variables sin usar, nombres que ocultan otros, …

```haskell
signo :: Int -> String
signo n | n > 0 = "positivo"
        | n < 0 = "negativo"
```

```
ghci> :set -Wall
ghci> :load signo.hs
...
signo.hs:2:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘signo’: Patterns of type ‘Int’ not matched: _
...
ghci> signo 0
"*** Exception: signo.hs:(2,1)-(3,28): Non-exhaustive patterns in function signo
```

`trace`

- Para usar `trace` es necesario:

```
import Debug.Trace
```

- `trace :: String -> a -> a`
  - `trace mensaje e` devuelve `e`. Cuando se evalúa, escribe `mensaje`.

```
fact :: Integer -> Integer
fact 0 = 1
fact n = trace ("fact " ++ show n) (n * fact (n - 1))
```

```
ghci> fact 3
fact 3
fact 2
fact 1
6
```

- El mensaje aparece cuando se evalúa la expresión, no cuando se llama a la función.

```
ghci> fst (1, trace "segundo" 2)
1
```

El depurador de `ghci`

- Este es el contenido de `suma.hs`, que suponemos ya cargado en `ghci`:

```
suma :: [Int] -> Int
suma [] = 0
suma (x:xs) = x + suma xs
```

- Este es un ejemplo de uso del depurador:

```
ghci> :break suma
Breakpoint 0 activated at suma.hs:2:11
Breakpoint 1 activated at suma.hs:3:15-25
ghci> suma [1, 2, 3]
Stopped in Main.suma, suma.hs:3:15-25
_result :: Int = _
x :: Int = 1
xs :: [Int] = [2,3]
[suma.hs:3:15-25] ghci> :continue
Stopped in Main.suma, suma.hs:3:15-25
_result :: Int = _
x :: Int = 2
xs :: [Int] = [3]
[suma.hs:3:15-25] ghci> :delete *
[suma.hs:3:15-25] ghci> :continue
6
```

Ejercicios

1. Queremos que la función `posicion` de abajo dé la posición de la primera aparición de un elemento en una lista, empezando en 0: `posicion 'c' "abcd"` ⇝ `2`. Sin embargo, `posicion 'c' "abcd"` ⇝ `3`, y `posicion 'z' "abcd"` termina con un error poco claro. Usa `trace` y `-Wall` para encontrar los fallos y mejora la definición.

```
posicion :: Eq a => a -> [a] -> Int
posicion x (y:ys) | x == y    = 1
                  | otherwise = 1 + posicion x ys
```

2. Predice qué escribe `ghci` para estas dos órdenes.

```
ghci> length [trace "a" 1, trace "b" 2]
ghci> sum [trace "a" 1, trace "b" 2]
```